

GreenStep Employee Sustainability Challenge

Technical Design Document

Oli Gurmessa Hadi Shaar Naisha Srivastav Nathanael Agbemadon

CISC480 — Senior Capstone

Instructor: Dr. Jimenez • Spring 2026

Submitted: May 22, 2026

Contents

1	System Overview / Proposed Solution	2
2	Architecture Pattern	2
2.1	Pattern: Modular Monolith (Layered)	2
2.2	Design Decisions and Justifications	2
3	Architectural Views	3
3.1	Logical View	3
3.2	Deployment View	3
3.3	Data-Flow View	3
4	C4 Model Diagrams	6
4.1	Level 1 — Context	6
4.2	Level 2 — Container	6
4.3	Level 3 — Component	6
4.4	Level 4 — Code (Entity-Relationship)	6
5	UML Diagrams	6
5.1	Class / Data-Model Diagram	6
5.2	Sequence Diagrams	9
5.3	Component Diagram	9
6	Data Design and Representation	9
7	Algorithm Explanations	10
7.1	Dense Rank with Tie Handling (Leaderboards)	10
7.2	Streak Calculation (on each action)	10
7.3	Duplicate-Submission Guard (10s window + serializable txn)	11
7.4	Unique Challenge-Code Generation	11
8	Deployment Plan	11

1 System Overview / Proposed Solution

GreenStep Employee is a web-based, gamified sustainability tracking platform designed for workplace use. Employees log sustainable actions across five categories—Transport, Water, Energy, Recycling, and Food—earning points, streaks, and badges that contribute to team and individual leaderboards. The platform replaces manual Excel-based tracking with a real-time, low-friction digital system.

The solution is delivered as a single web application accessible from any browser. It requires no installation and is hosted on Vercel’s free tier, keeping it viable within the project’s zero-budget constraint. Participation is organized into *challenges*: every user is auto-enrolled in a permanent global challenge and may join additional, code-gated challenges.

2 Architecture Pattern

2.1 Pattern: Modular Monolith (Layered)

The system follows a **Modular Monolith** pattern organized into three logical layers—Presentation, Application, and Data—all deployed as a single Next.js 15 application (Figure 1). The **Application layer** is not a separate server: it comprises the Next.js *API route handlers*, *server actions* (`app/actions/challenge.ts`), and the *Clerk middleware* that authenticates every non-public request before it reaches a route.

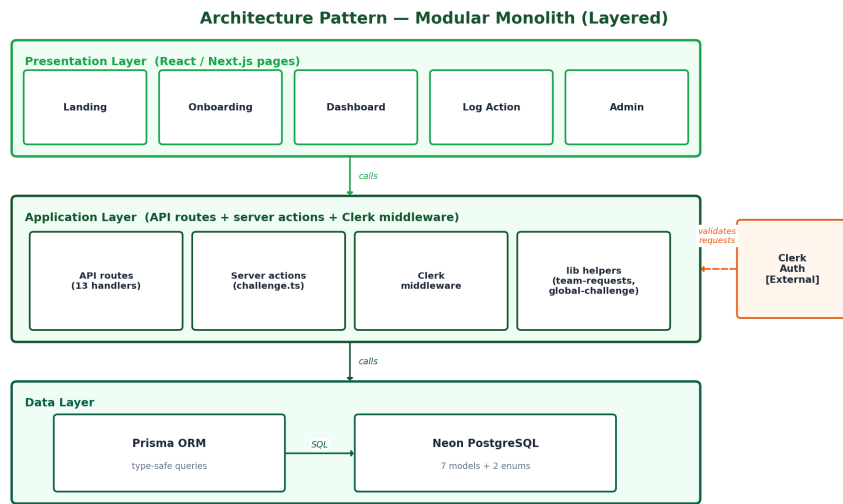


Figure 1: Modular-monolith layered architecture. Presentation (React pages) calls the Application layer (API routes, server actions, Clerk middleware), which calls the Data layer (Prisma ORM over Neon PostgreSQL). Clerk is a cross-cutting external dependency that validates every request.

2.2 Design Decisions and Justifications

Table 1 links each major decision to a project requirement or constraint.

Table 1: Design decisions and their justifications.

Decision	Justification	Requirement/Constraint
Modular monolith over microservices	Single deployment unit; no inter-service networking or orchestration.	4-person team, one semester, no DevOps budget.
Next.js 15 for UI and API	Co-locating frontend, API routes, and server actions removes a separate backend, CORS, and a second pipeline.	Simplicity; single Vercel deployment; free tier.
Clerk for authentication	Production-grade auth, sessions, and JWTs without building or securing a custom system; admin via <code>publicMetadata</code> .	Security requirement; no auth-infra budget.
Prisma + Neon PostgreSQL	Type-safe access over a relational 7-model schema; serverless Postgres with a free tier and no maintenance.	Relational data; zero budget; low maintenance.
Vercel hosting	Zero-config build and deploy on push to <code>main</code> .	Free tier; CI/CD without extra tooling.
Five action categories (enum)	Mirrors the partner’s existing sustainability programme domains.	Partner requirement.

3 Architectural Views

3.1 Logical View

Figure 2 maps the three layers to concrete modules: the `app/` pages (Presentation); the `app/api/` route handlers, `app/actions/` server actions, `middleware.ts`, and `lib/` helpers (Application); and `lib/db.ts` with `prisma/schema.prisma` over Neon (Data).

3.2 Deployment View

Figure 3 shows the runtime nodes. The browser talks only to the Next.js app on Vercel; the app talks to Clerk (HTTPS) and Neon (TCP/PostgreSQL via Prisma).

3.3 Data-Flow View

Figure 4 traces a single request—logging an action—from the browser through authentication, challenge resolution, the serializable transaction, and the streak update.

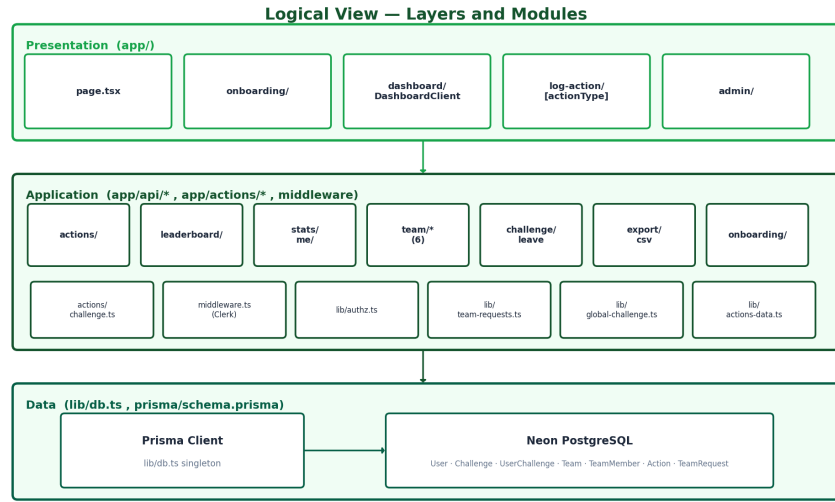


Figure 2: Logical view: the modules that make up each layer of the application.

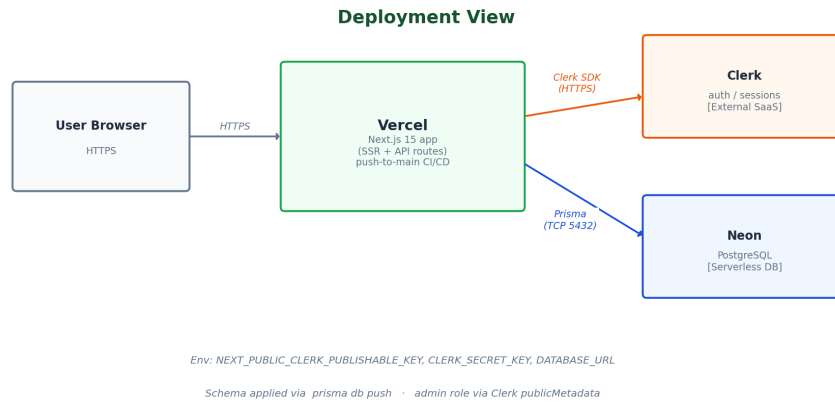


Figure 3: Deployment view: Vercel (Next.js app), Neon (PostgreSQL), and Clerk (auth) as separate nodes, with the required environment variables.

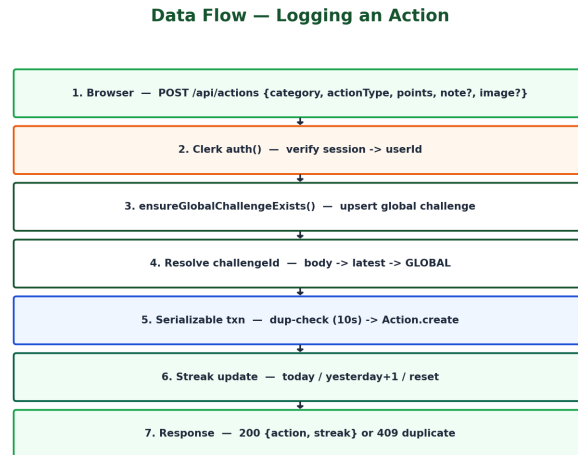


Figure 4: Data-flow view for “log an action,” from `POST /api/actions` to the response.

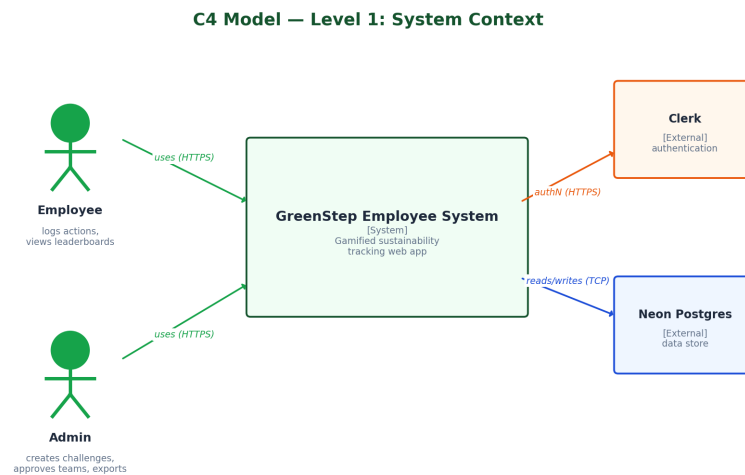


Figure 5: C4 Context. The GreenStep system used by Employee and Admin actors, delegating authentication to Clerk and persistence to Neon PostgreSQL.

4 C4 Model Diagrams

4.1 Level 1 — Context

Figure 5 gives the highest-level view: employees log actions and view leaderboards; admins create challenges, approve teams, and export data.

4.2 Level 2 — Container

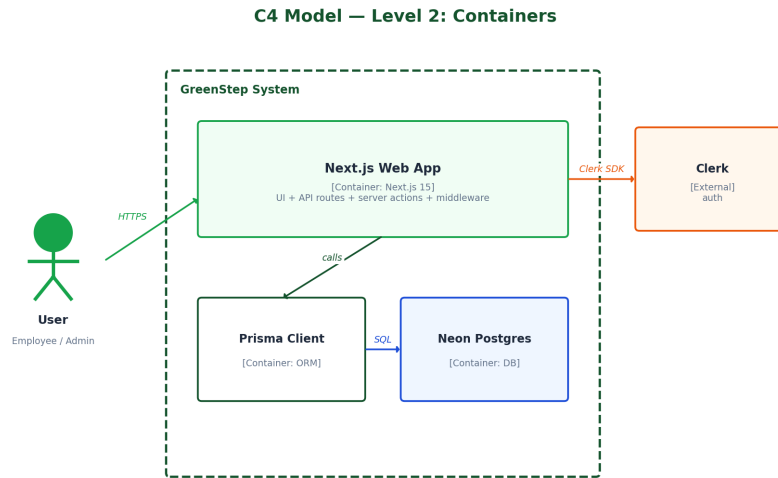


Figure 6: C4 Container. One Next.js container serves UI, API routes, server actions, and middleware; it uses the Prisma client to reach Neon and the Clerk SDK to validate sessions.

4.3 Level 3 — Component

As shown in Figure 7, every inbound request first passes through the Clerk middleware. Authenticated requests reach a page or one of the API route handlers; all database access is centralized through the Prisma client.

4.4 Level 4 — Code (Entity-Relationship)

Figure 8 is the authoritative data model and is reused as the UML class/data-model diagram in Section 5.

5 UML Diagrams

5.1 Class / Data-Model Diagram

The class/data-model view is the seven-entity diagram in Figure 8. The entities are `User`, `Challenge`, `UserChallenge`, `Team`, `TeamMember`, `Action`, and `TeamRequest`. `UserChallenge` and `TeamMember` are join tables implementing the many-to-many relations `User↔Challenge` and `User↔Team`. Two enums constrain string fields: `ActionCategory` and `TeamRequestStatus`.

C4 Model — Level 3: Components (inside the Next.js container)

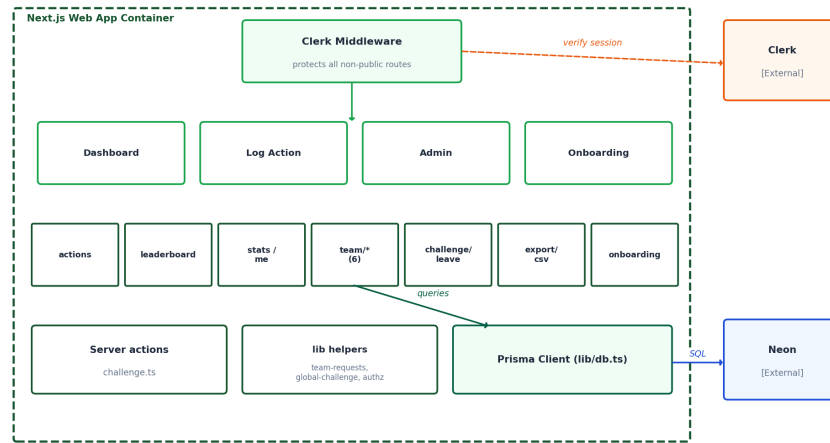


Figure 7: C4 Component (inside the Next.js container): Clerk middleware; the UI pages; the thirteen API route handlers; the `challenge.ts` server actions; the `lib` helpers (`team-requests`, `global-challenge`, `authz`); and the Prisma client, which is the single point of database access.

Data Model — 7 Entities (Prisma schema)

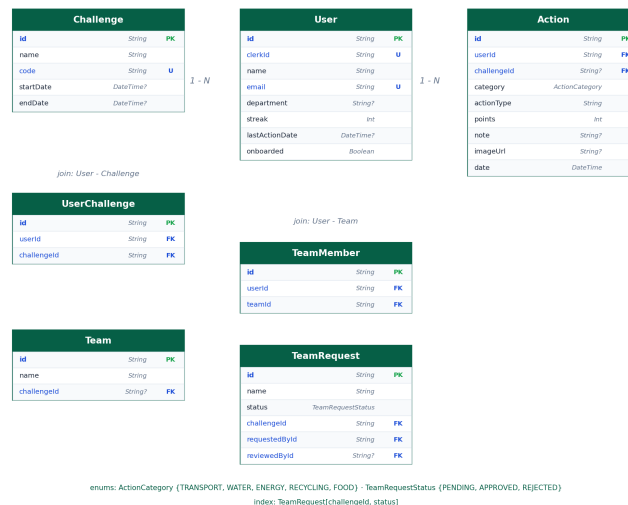


Figure 8: C4 Code level shown as the entity-relationship model of the seven Prisma entities and their keys and relations.

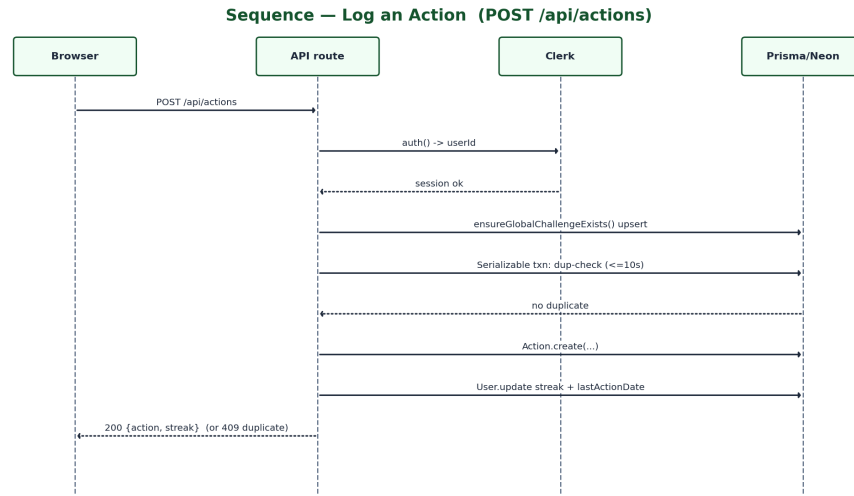


Figure 9: Sequence: logging an action. The route authenticates via Clerk, ensures the global challenge exists, runs a serializable transaction (duplicate check then create), and updates the streak.

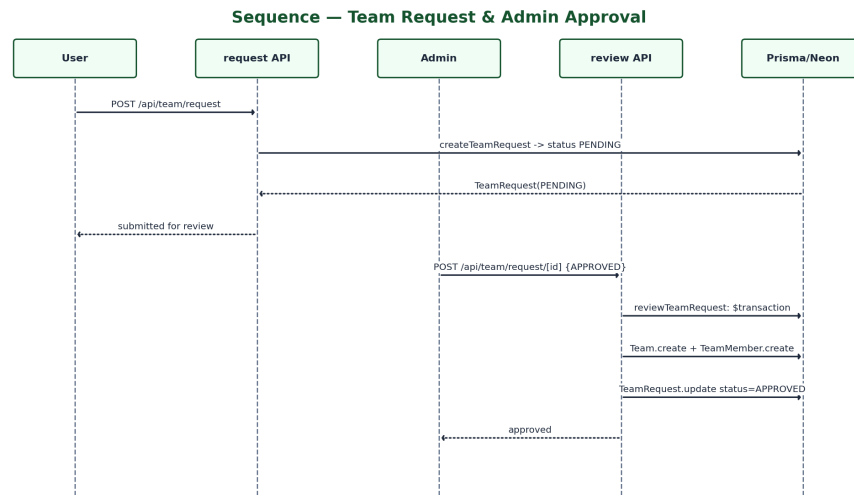


Figure 10: Sequence: team request and admin approval. A user submits a **PENDING** request; an admin approves it, triggering a **\$transaction** that creates the Team and TeamMember and marks the request **APPROVED**.

5.2 Sequence Diagrams

Figure 9 and Figure 10 detail the two most safety-critical flows: action logging (with its duplicate guard) and the admin-gated team-creation workflow.

5.3 Component Diagram

The component view of the system—the Next.js internals and the external Clerk and Neon dependencies—is given by the C4 Level 3 diagram, Figure 7.

6 Data Design and Representation

The schema (`prisma/schema.prisma`) defines seven models. Primary keys are `cuid` strings. Tables 2–3 list the fields; “U” marks a unique constraint and “FK” a foreign key.

Table 2: Core entities: User, Challenge, and Action.

Model	Key fields	Notable fields / relations
User	id (PK), clerkId (U), email (U)	name, department?, streak (default 0), lastActionDate?, onboarded (default false); has many Action, UserChallenge, TeamMember, TeamRequest.
Challenge	id (PK), code (U)	name, startDate?, endDate?; has many UserChallenge, Team, Action, TeamRequest.
Action	id (PK)	userId (FK), challengeId? (FK), category (ActionCategory), actionType, points (default 0), note?, imageUrl? (Text), date.

Table 3: Team, TeamRequest, and the two join tables.

Model	Key fields	Notable fields / relations
Team	id (PK)	name (not unique — names may repeat across challenges), challengeId? (FK); has many TeamMember.
UserChallenge	id (PK); @@unique([userId, challengeId])	userId (FK), challengeId (FK) — enrollment join table.
TeamMember	id (PK); @@unique([userId, teamId])	userId (FK), teamId (FK) — membership join table.
TeamRequest	id (PK); @@index([challengeId, status])	name, message?, status (TeamRequestStatus, default PENDING), challengeId (FK), requestedById (FK), reviewedById? (FK), reviewedAt?.

Enumerations. ActionCategory = { TRANSPORT, WATER, ENERGY, RECYCLING, FOOD }; TeamRequestStatus = { PENDING, APPROVED, REJECTED }. The [challengeId, status] index on TeamRequest accelerates the admin queue, which filters pending requests by challenge.

7 Algorithm Explanations

7.1 Dense Rank with Tie Handling (Leaderboards)

Items are sorted by value (descending), breaking ties by name. Ranks increment only when the value changes, so tied entries share a rank (1, 2, 2, 3).

```
sort items by value desc, then name asc
rank <- 0; prev <- null
for each item:
  if prev is null or item.value != prev:
    rank <- rank + 1
    prev <- item.value
  item.rank <- rank
```

7.2 Streak Calculation (on each action)

```
todayStart <- midnight(now)
yesterdayStart <- todayStart - 1 day
newStreak <- 1
if user.lastActionDate exists:
  last <- midnight(user.lastActionDate)
  if last == todayStart: newStreak <- user.streak # already today
  elif last == yesterdayStart: newStreak <- user.streak + 1 # consecutive
```

```

else:
    newStreak <- 1
    # gap -> reset
update user.streak <- newStreak, lastActionDate <- now

```

7.3 Duplicate-Submission Guard (10 s window + serializable txn)

To prevent double-logging from rapid taps or retries, creation runs inside a `SERIALIZABLE` transaction that first checks for an identical action within the last ten seconds.

```

cutoff <- now - 10 seconds
BEGIN TRANSACTION (SERIALIZABLE):
    existing <- Action where userId, challengeId, category, actionType,
        points, note, imageUrl all match AND createdAt >= cutoff
    if existing: throw DUPLICATE          # -> HTTP 409
    else:      create Action
COMMIT
# a Prisma P2034 serialization failure is also returned as HTTP 409

```

7.4 Unique Challenge-Code Generation

```

do:
    code <- random base36 string, 6 chars, upper-cased
while a Challenge with that code already exists

```

8 Deployment Plan

- **Hosting / CI-CD.** The app is hosted on **Vercel**. Building and deploying are triggered by pushing to `main` (or via `vercel --prod`); Vercel installs dependencies, runs `next build`, and serves the result.
- **Database.** **Neon** serverless PostgreSQL. The schema is applied with `prisma db push`; the project intentionally has no migrations folder for this phase.
- **Authentication.** **Clerk**. Administrative privileges are granted by setting a Clerk user's `publicMetadata.role` to `"admin"`.
- **Required environment variables.** `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`, `CLERK_SECRET_KEY`, and `DATABASE_URL` (plus the Clerk sign-in/up redirect URLs).

References

- [1] Vercel, “Next.js Documentation,” 2024. [Online]. Available: <https://nextjs.org/docs>
- [2] Clerk, “Clerk Documentation,” 2024. [Online]. Available: <https://clerk.com/docs>
- [3] Prisma, “Prisma ORM Documentation,” 2024. [Online]. Available: <https://www.prisma.io/docs>
- [4] Neon, “Neon Serverless Postgres Documentation,” 2024. [Online]. Available: <https://neon.tech/docs>
- [5] S. Brown, “The C4 model for visualising software architecture,” 2024. [Online]. Available: <https://c4model.com/>
- [6] E. Adekoya, K. Alkhaleefa, and J. Dunlap, “Employee Sustainability Challenge App: Final Project Documentation,” CISC489 Topics in Human-Computer Interaction, University of St. Thomas, Dec. 2025.